

Nexus AI

Technical Implementation Report

A modular AI operating system for knowledge workers: long-term memory, self-improving retrieval-augmented generation, multi-agent routing, code intelligence, vision, and live web research.

Repository: github.com/usmanwajid09/Nexus-AI

Stack: Python / FastAPI / LangGraph / PostgreSQL+pgvector / Claude API
63 source files | 53 unit tests | 11 API endpoints | CI on Python 3.12 & 3.13
July 2026

1. What Nexus AI Is

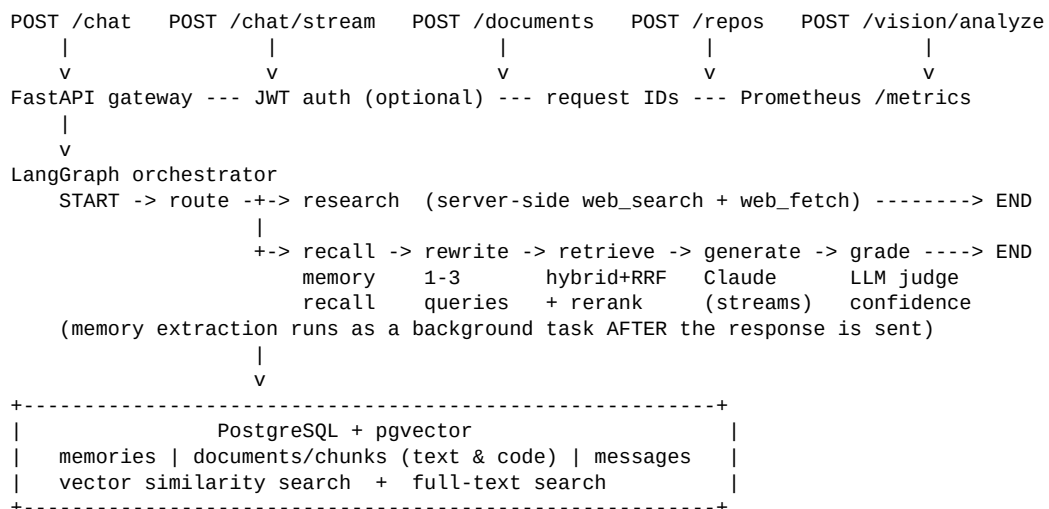
Nexus AI is a backend platform that behaves like an AI assistant with an operating system's ambitions: it **remembers** facts about the user across sessions, **retrieves** answers from the user's own documents and source code, **routes** each request to the right specialist pipeline, **grades its own answers** for trustworthiness, **researches the live web** when a question needs current information, and **understands images**. It is built as one coherent service, not a collection of demos: every request flows through a single graph-based orchestrator backed by one PostgreSQL database.

The guiding engineering principle was **a vertical slice over horizontal sprawl**: instead of building ten half-finished modules, every phase shipped a thin thread that works end to end, with each later capability added as a node or branch in the existing pipeline. The second principle was **graceful degradation**: every auxiliary intelligence step (query rewriting, reranking, grading, memory extraction) falls back to simpler behavior on failure and can never break the user's request.

Capabilities at a glance

- **Long-term memory** - every chat turn is mined by an LLM into episodic, semantic, and procedural memories; relevant memories are recalled into future conversations.
- **Self-improving RAG** - conversational questions are rewritten into search queries, retrieved with hybrid vector+keyword search fused by Reciprocal Rank Fusion, reranked by an LLM, and the final answer is graded for groundedness with a confidence score.
- **Multi-agent routing** - a router classifies each message as general / code / research and sends it down the matching branch of the orchestration graph.
- **Code intelligence** - local repositories are ingested with definition-boundary chunking (12 languages); code questions retrieve only code and answers propose changes as unified diffs grounded in the retrieved source.
- **Live web research** - a research agent uses Claude's server-side `web_search` and `web_fetch` tools and cites source URLs.
- **Vision** - an endpoint analyzes uploaded images (screenshots, diagrams, charts) and can ingest the analysis back into the knowledge base.
- **Streaming** - a Server-Sent Events endpoint streams the answer token by token with progress events (route, sources, confidence).
- **Production hygiene** - optional JWT auth, Prometheus metrics, Grafana, structured request logging, an offline evaluation harness, unit tests, and CI.

2. High-Level Architecture



The system has four layers. The **gateway** (FastAPI) handles HTTP, auth, validation, logging, and metrics. The **orchestrator** (LangGraph) is a state machine that decides what happens for each message. The **intelligence services** (LLM provider, embeddings provider, reranker) sit behind small Python interfaces so any of them can be swapped in one file. The **storage layer** is a single PostgreSQL database that serves relational data, vector similarity search, and keyword full-text search simultaneously.

3. Technology Stack - What Each Piece Is, Why It Was Chosen, How It Is Used

3.1 Python 3.12+ with asyncio

What it is: asyncio is Python's built-in cooperative concurrency framework. Instead of one thread per request, a single event loop interleaves many tasks that spend most of their time waiting on I/O (network calls, database queries).

Why here: an AI backend is almost entirely I/O-bound - every request waits on the LLM API and the database. Async lets one process serve many concurrent chats cheaply.

How used: the entire stack is async end to end: FastAPI endpoints, SQLAlchemy async sessions, the Anthropic SDK's AsyncAnthropic client, and LangGraph's async node functions. Background memory writing uses `asyncio.create_task` with a strong-reference set (a well-known asyncio pitfall: tasks without a live reference can be garbage collected mid-flight).

3.2 FastAPI (with Uvicorn and Pydantic)

What it is: FastAPI is the leading modern Python web framework. It builds on Starlette (async HTTP toolkit) and Pydantic (data validation via Python type hints), and auto-generates interactive OpenAPI documentation at `/docs`. Uvicorn is the ASGI server that actually runs it.

Why here: native async support, request/response validation for free from type hints, dependency injection (used for the auth guard), and self-documenting APIs - the de facto standard for Python AI services.

How used: eleven endpoints defined in `src/nexus/api/main.py` with Pydantic schemas in `schemas.py`. A middleware stamps every request with a request ID, logs method/path/status/latency, and records a Prometheus histogram. The lifespan hook initializes the database, the LLM provider, the embedder, and compiles the graph once at startup. `StreamingResponse` powers the SSE endpoint; `UploadFile/Form` handle image uploads.

3.3 LangGraph (and how it relates to LangChain)

What LangChain is: LangChain is the most widely used framework for building LLM applications. It provides abstractions for models, prompts, tools, document loaders, and retrievers, and originally popularized 'chains' - fixed, linear sequences of LLM calls. Its weakness for complex agents is that real agent behavior is not linear: it branches, loops, and maintains state.

What LangGraph is: LangGraph is a lower-level orchestration library from the LangChain team, built specifically for stateful, multi-step, possibly-cyclic agent workflows. You model your application as a **state machine**: a typed state object (a TypedDict) flows through a directed graph of nodes; each node is a function that reads state and returns a partial update; edges (including conditional edges) decide what runs next. The graph is compiled once and then invoked or streamed per request. This is the same architectural pattern used by production agent systems generally.

Why LangGraph over classic LangChain chains: the router needs conditional branching (general vs code vs research), the design needed room to grow into loops (retry-on-low-confidence is a natural future edge), and LangGraph's streaming modes let token deltas flow out of a node mid-execution. Plain chains cannot express any of that cleanly. LangChain itself is used only indirectly (langchain-core is a dependency of LangGraph).

How used: `src/nexus/orchestrator/graph.py` defines `NexusState` and seven nodes: `route`, `research`, `recall`, `rewrite`, `retrieve`, `generate`, `grade`. `add_conditional_edges` on the router picks the branch. The `generate` node obtains `LangGraph`'s custom stream writer (`get_stream_writer`) and emits token deltas; when the graph is invoked non-streaming these writes are dropped harmlessly, so both `/chat` and `/chat/stream` share exactly one pipeline.

3.4 Anthropic Claude API (model: claude-opus-4-8)

What it is: the Claude API exposes Anthropic's large language models over a Messages endpoint. Beyond plain text generation it provides: **adaptive thinking** (the model decides when and how much to reason internally before answering), **structured outputs** (the response is constrained to match a supplied JSON schema), **native vision** (images as input), **server-side tools** (Anthropic runs web search / web fetch on their infrastructure and returns results inside the same API call), and **streaming** (tokens over Server-Sent Events).

Why here: one API covers five needs that would otherwise be five systems: generation, JSON-mode classification, image understanding, web research, and streaming. Structured outputs in particular eliminate an entire class of 'the model returned almost-JSON' bugs.

How used: `src/nexus/llm/anthropic_provider.py` implements the provider interface with five methods: `complete` (chat, adaptive thinking), `complete_stream` (SDK streaming helper, yields text deltas), `complete_json` (structured outputs with strict schemas - `additionalProperties:false`, required fields), `complete_vision` (base64 image block + question), and `research` (server-side `web_search_20260209` + `web_fetch_20260209` tools with `pause_turn` continuation - if the server-side tool loop pauses, the partial assistant turn is appended and the request re-sent so it resumes). Refusal stop reasons are handled explicitly, and every call records token usage into Prometheus counters. Every machine-read LLM output (routing, rewriting, reranking, grading, memory extraction) goes through `complete_json`.

3.5 PostgreSQL + pgvector

What PostgreSQL is: the most trusted open-source relational database - ACID transactions, rich SQL, and a mature extension ecosystem.

What pgvector is: a PostgreSQL extension that adds a VECTOR column type plus similarity operators (cosine distance, inner product, L2) and approximate-nearest-neighbor indexes (HNSW, IVFFlat). It turns Postgres into a competent vector database.

Why here instead of a dedicated vector DB (Qdrant, Pinecone, Weaviate): the original plan called for Postgres + Qdrant + Neo4j + Redis + Kafka. That is five systems to operate, back up, and keep consistent, before the product works. One Postgres serves relational data (conversations, messages), vector search (memories, chunks), and keyword full-text search (tsvector) in a single transaction domain. The storage access sits behind small functions, so a dedicated vector DB can be swapped in when a measured limit (collection size, latency, QPS) actually demands it - not before.

How used: five tables (conversations, messages, memories, documents, chunks). memories and chunks carry a `Vector(1024)` embedding column queried with `cosine_distance`; chunks are additionally searched with `to_tsvector/plainto_tsquery` ranked by `ts_rank`. Documents carry a `kind` column ('text' or 'code') so the code route can filter retrieval to source code. `init_db()` creates the vector extension and tables at startup (Alembic migrations are deliberately deferred until the schema stabilizes).

3.6 SQLAlchemy 2.0 (async) + psycopg 3

What it is: SQLAlchemy is Python's standard ORM and SQL toolkit; the 2.0 API uses typed, declarative models (`Mapped[...]` annotations) and `select()` statements. `psycopg 3` is the modern PostgreSQL driver with first-class async support.

Why here: typed models catch schema mistakes early; the async engine matches the async stack. `psycopg3` was chosen over `asyncpg` deliberately - it has a pure-Python fallback and a better story on Windows and brand-new Python versions (this project was developed on Python 3.14).

How used: models in `src/nexus/db/models.py`; a cached async engine and session factory in `session.py`; every unit of work opens a short-lived session from the factory. pgvector's SQLAlchemy integration provides the Vector column type and `cosine_distance` operator directly in ORM queries.

3.7 Voyage AI embeddings (with a pure-Python fallback)

What embeddings are: an embedding model maps text to a high-dimensional vector (here 1024 numbers) such that semantically similar texts land near each other. Similarity between a question and a stored chunk is measured with cosine similarity, which is what makes semantic search possible.

Why Voyage: Voyage AI is Anthropic's recommended embeddings partner (Anthropic does not ship an embeddings API); voyage-3.5 supports asymmetric embedding - `input_type='document'` when indexing and `input_type='query'` when searching - which measurably improves retrieval.

How used: behind an `EmbeddingProvider` interface with two implementations: `VoyageEmbeddings` (production) and `HashingEmbeddings` (development/tests) - a deterministic bag-of-words 'feature hashing' vectorizer: each token is MD5-hashed to pick a bucket and a sign, and the vector is L2-normalized. It captures lexical overlap only (no semantics) but requires zero API keys and zero native code, so the whole system - including the eval harness - runs offline. The provider is selected by the `EMBEDDING_PROVIDER` environment variable.

3.8 PyJWT (JSON Web Tokens)

What JWT is: a signed token format for stateless authentication. The server signs a JSON payload (subject, issued-at, expiry) with a secret (HS256 = HMAC-SHA256); any later request presenting the token can be verified without a session store.

How used: `src/nexus/api/auth.py` implements `create_token` / `verify_token` and a FastAPI dependency `require_auth` applied to every data endpoint. If `AUTH_SECRET` is unset the API runs in explicit single-user dev mode; if set, requests need 'Authorization: Bearer <token>' (minted by `scripts/make_token.py`). Startup warns if the secret is shorter than 32 characters. Tokens expire (default 60 minutes).

3.9 Prometheus + Grafana

What they are: Prometheus is the standard open-source metrics system - services expose a `/metrics` text endpoint, Prometheus scrapes it on an interval and stores time series. Grafana is the dashboarding UI that queries Prometheus.

How used: `src/nexus/observability.py` defines a request-latency histogram (method/path/status), LLM call and token counters (by call kind and direction), a retrieval-size histogram, and an answer-confidence histogram - so RAG quality is visible on a dashboard over time, not just in logs. `prometheus-client`'s ASGI app is mounted at `/metrics`. `docker compose --profile observability` brings up Prometheus (scraping the API via `host.docker.internal`) and Grafana.

3.10 Docker + docker-compose

What it is: containers package a service with its exact dependencies; `docker-compose` declares a multi-service development stack in one YAML file.

How used: the compose file runs `pgvector/pgvector:pg17` (Postgres with the extension pre-installed) with a named volume and healthcheck, plus an optional 'observability' profile for Prometheus and Grafana. Kubernetes was deliberately rejected for this stage - operational complexity before users is cost without benefit.

3.11 pytest, ruff, and GitHub Actions

What they are: `pytest` is Python's dominant test framework; `ruff` is an extremely fast linter; GitHub Actions runs workflows (install, lint, test) on every push and pull request.

How used: 53 unit tests target the pure logic: both chunkers, RRF fusion, every LLM-output parser (routing, rewriting, reranking, grading, memory extraction), eval metrics, JWT round-trips, SSE formatting, and the provider streaming default. The testing philosophy: anything that can be a pure function is one, and that is where

the tests live - no network, no database, no API keys, so the suite runs in about one second. CI runs ruff + pytest on Python 3.12 and 3.13; the README carries the status badge.

3.12 Server-Sent Events (SSE)

What it is: a simple one-directional streaming protocol over plain HTTP: the server keeps the response open and writes 'event: name / data: json' frames. Browsers consume it natively via EventSource. It is simpler than WebSockets when the client only listens - which is exactly the chat-streaming case.

How used: POST /chat/stream returns frames in a fixed order: meta (conversation id), route, sources, delta (one per token group), grade (confidence + unsupported claims), done (the complete payload). Inside, `graph.astream(..., stream_mode=['updates','custom'])` interleaves node-completion updates with the generate node's token deltas.

4. Core Concepts You Should Be Able to Explain

4.1 RAG (Retrieval-Augmented Generation)

An LLM only knows its training data. RAG fixes that: at question time, retrieve the most relevant snippets from the user's own corpus, place them in the prompt, and instruct the model to answer from them with citations. Compared to fine-tuning, RAG is instantly updatable (ingest a document and it is usable immediately), auditable (you can see which sources produced the answer), and far cheaper. Nexus's whole retrieve->generate->grade pipeline is a RAG system with extra quality stages.

4.2 Chunking

Documents are too large to embed whole, so they are split into chunks - the unit of retrieval. Chunk badly and retrieval fails no matter how good the models are. Nexus has two chunkers, both pure functions: the **text chunker** packs whole paragraphs up to ~1600 characters and hard-splits oversized paragraphs with a character overlap so boundary context is not lost; the **code chunker** splits source files at top-level definition boundaries (def/class/function/func...) per language family, so a chunk is usually a complete function rather than an arbitrary window.

4.3 Hybrid search (vector + keyword)

Vector search finds semantically similar text but can miss exact identifiers; keyword search (Postgres full-text: to_tsvector / plainto_tsquery / ts_rank) nails exact terms like 'make deploy' or an error code but misses paraphrases. Production retrieval systems run both. Nexus runs both per query - and because the rewrite step can produce up to three queries, retrieval may fuse up to six ranked lists.

4.4 Reciprocal Rank Fusion (RRF)

Vector scores (cosine similarity) and keyword scores (ts_rank) live on incomparable scales, so you cannot just add them. RRF sidesteps score normalization entirely by using only rank positions: each item scores $\text{sum}(1 / (k + \text{rank}))$ across every list it appears in, with $k=60$ (the standard constant that damps the top-rank dominance). An item found by both retrievers naturally rises to the top. Implemented as a ~15-line pure function (src/nexus/rag/fusion.py) with unit tests.

4.5 Query rewriting (multi-query retrieval)

Users ask conversational questions ('how does that work?') that make poor search queries. A structured-output LLM call rewrites the question into 1-3 self-contained search queries, resolving pronouns from conversation history. All queries retrieve independently and RRF fuses the results. Failure degrades to the original question.

4.6 Reranking

First-stage retrieval optimizes recall; a reranker then re-orders the candidates for precision. The classical tool is a cross-encoder (a small transformer scoring each query-passage pair), but that drags in torch and native binaries. Nexus uses a **listwise LLM reranker**: the fused top-2x candidates are shown to Claude, which returns a ranking of indices via structured outputs. A pure function repairs the permutation (drops out-of-range or duplicate indices, appends forgotten ones) so the result is always valid; failure degrades to fused order.

4.7 Groundedness grading (LLM-as-judge)

After answering, a second LLM call grades the answer against the retrieved context: a grounded_score of 0-100 (clamped and normalized to 0-1) plus a list of unsupported claims. The API returns this as 'confidence', and a Prometheus histogram tracks it over time - this is the hallucination alarm. The judge sees only the context and the answer, so it cannot be swayed by the model's confidence in itself.

4.8 Long-term memory (episodic / semantic / procedural)

The taxonomy mirrors cognitive science: **episodic** = events ('user finished the auth migration'), **semantic** = durable facts ('backend uses FastAPI'), **procedural** = how-to knowledge ('deploy with make deploy'). Implementation insight: these are labels in one table, not three storage engines. After every chat turn a background LLM call extracts memories via a strict JSON schema with explicit anti-hallucination rules ('an empty list is a good answer'); recall is cosine similarity over memory embeddings. Each recall updates `last_accessed_at` and `access_count` - the substrate for future decay/reinforcement without a schema change.

4.9 Multi-agent routing

Rather than a hierarchy of chatty agents, Nexus uses a single classifier node that picks one of three branches: general (memory + document RAG), code (RAG filtered to `kind='code'` with diff-proposing instructions), research (live web tools). Each specialist is a graph branch sharing the same state type - adding a specialist is one node plus one edge, not a new service.

4.10 Structured outputs + defensive parsing

Every LLM output consumed by code (never by a human) is doubly protected: the API constrains generation to a strict JSON schema, AND a pure parser validates the payload anyway - clamping scores, dropping invalid entries, defaulting to safe values. This 'trust but verify' pattern means a malformed model output can never produce an exception in the pipeline, only a graceful downgrade.

4.11 Offline evaluation (hit@k and MRR)

A golden set (`evals/golden.jsonl`) maps questions to the document that should answer them. The harness runs real retrieval and reports **hit@k** (was the right document in the top k?) and **MRR** (mean reciprocal rank - $1/\text{position of the first correct result}$, averaged). `scripts/seed_demo.py` seeds demo documents that match the golden set, so a fresh clone gets a baseline with no API keys. This is the regression gate: run it before and after touching chunking, embeddings, rewriting, fusion, or reranking.

5. Life of a Request (step by step)

POST /chat with {"message": "How does auth work?"}:

- **1. Auth** - require_auth dependency: no-op in dev mode, otherwise JWT verification.
- **2. Conversation setup** - find or create the Conversation row; load the last 20 messages as history; persist the user message.
- **3. route** - structured-output classification -> 'general' (not web, not code).
- **4. recall** - embed the question, cosine-search the memories table (top 5), update access stats.
- **5. rewrite** - LLM produces up to 3 self-contained search queries.
- **6. retrieve** - per query: vector top-12 + keyword top-12; RRF-fuse all lists; take 12 candidates; LLM reranker orders them; keep top 6 chunks with document titles.
- **7. generate** - system prompt assembled from memory block + numbered context excerpts + citation instructions; Claude streams the answer (deltas dropped in non-streaming mode).
- **8. grade** - LLM judge scores groundedness (0-1) and lists unsupported claims.
- **9. Persist + respond** - assistant message saved; response includes answer, route, recalled memories, sources, confidence, unsupported claims.
- **10. Background memorize** - after the response is sent, an LLM extraction call mines the turn for memories and stores them with embeddings; failures are logged, never raised.

POST /chat/stream runs the same graph via astream and emits SSE frames: meta -> route -> sources -> delta (per token group) -> grade -> done. The research route emits its full answer as one delta (server-side tool turns do not stream token-by-token).

6. Module-by-Module File Map

File	Responsibility
src/nexus/config.py	All settings via pydantic-settings; reads .env; feature toggles (rewrite/rerank/grading), auth, limits.
src/nexus/_compat.py	Pure-Python uuid_utils stand-in for machines whose security policy blocks the native DLL langchain-core imports.
src/nexus/llm/base.py	LLMProvider interface: complete, complete_json, complete_stream (default: one chunk), complete_vision, research.
src/nexus/llm/anthropic_provider.py	Claude implementation: adaptive thinking, structured outputs, SDK streaming, vision, server-side web tools with pause_turn handling, refusal handling, usage metrics.
src/nexus/embeddings/(base voyage hashing).py	EmbeddingProvider interface; Voyage voyage-3.5 (document/query input types); deterministic hashing fallback.
src/nexus/db/models.py	SQLAlchemy models: Conversation, Message, Memory (type enum, embedding, access stats), Document (kind), Chunk (position, embedding).
src/nexus/db/session.py	Cached async engine/session factory; init_db creates the vector extension + tables.
src/nexus/rag/chunker.py	Paragraph-packing text chunker with overlap on hard splits; overlap clamps instead of failing when max_chars is small.
src/nexus/rag/fusion.py	Reciprocal Rank Fusion (pure).
src/nexus/rag/retriever.py	Multi-query hybrid retrieval: vector + full-text per query, RRF fusion, optional kind filter, optional reranker.
src/nexus/rag/rewrite.py	Query rewriting via structured outputs + pure parser.
src/nexus/rag/rerank.py	Reranker interface; listwise LLM reranker; permutation repair (pure).
src/nexus/rag/grading.py	Groundedness judge; Grade dataclass; clamped parser (pure).

File	Responsibility
src/nexus/rag/ingest.py	Document ingestion: chunk, batch-embed, store.
src/nexus/memory/extractor.py	Memory extraction prompt + strict schema.
src/nexus/memory/parsing.py	Defensive validation of extraction payloads (pure).
src/nexus/memory/store.py	add_memories / recall (cosine search + access stats).
src/nexus/memory/writer.py	Background memorize_turn; swallows and logs failures.
src/nexus/orchestrator/router.py	3-way route classification + pure parser.
src/nexus/orchestrator/graph.py	The LangGraph state machine: 7 nodes, conditional branching, streaming deltas via custom stream writer.
src/nexus/code/chunker.py	Definition-boundary code chunker, 12 languages (pure).
src/nexus/code/ingest.py	Repo walker: skip lists, size caps, per-file chunk+embed, kind='code' documents.
src/nexus/evals/metrics.py	hit@k, reciprocal rank, mean (pure).
src/nexus/observability.py	Prometheus metrics + /metrics ASGI app.
src/nexus/api/auth.py	JWT create/verify + require_auth dependency; dev-mode off-switch.
src/nexus/api/streaming.py	SSE frame formatting (pure).
src/nexus/api/schemas.py	Pydantic request/response models.
src/nexus/api/main.py	FastAPI app: lifespan wiring, middleware, all endpoints, background task management.
scripts/seed_demo.py	Idempotent demo data seeding aligned with the golden set.
scripts/run_evals.py	Retrieval evaluation harness (hit@k, MRR).
scripts/make_token.py	JWT minting utility.
tests/ (10 files)	53 unit tests over all pure logic.
.github/workflows/ci.yml	CI: ruff + pytest on Python 3.12/3.13.
docker-compose.yml / ops/prometheus.yml	Postgres+pgvector; optional Prometheus + Grafana profile.

7. Phase-by-Phase Build Log

Phase	Delivered
1 - Core slice	FastAPI gateway, LangGraph pipeline (recall->retrieve->generate), memory engine with LLM extraction, hybrid RAG with RRF, provider interfaces, docker-compose Postgres, first 19 tests. Bug found by tests: default overlap larger than a caller's max_chars crashed the chunker - fixed by clamping.
2 - Self-improving RAG	Query rewriting (multi-query), listwise LLM reranking, groundedness grading with confidence in the API response, background memory writer (zero added latency), JWT auth with dev mode, eval harness (hit@k/MRR) + golden set, feature toggles.
3 - Multi-agent + observability	Router node with conditional graph edges; research agent on server-side web_search/web_fetch with pause_turn continuation; Prometheus metrics (latency, LLM calls/tokens, retrieval size, confidence) + Grafana compose profile.
4 - Code intelligence	POST /repos walks a local checkout (skip lists, size caps), chunks at definition boundaries across 12 languages, stores kind='code'; code route retrieves only code and proposes unified diffs grounded in retrieved source.

Phase	Delivered
5 - Vision + web	POST /vision/analyze on Claude's native multimodality with optional ingestion of the analysis into the knowledge base; research route covers live-web needs.
Post-phase round	SSE streaming endpoint (/chat/stream) with token deltas through LangGraph's custom stream channel; conversation management endpoints (list/detail/delete); GitHub Actions CI + README badge; demo seeding aligned to the golden set.

8. Design Decisions and Trade-offs (the interview gold)

Decision	Reasoning / trade-off
One Postgres instead of Postgres+Qdrant+Neo4j+Redis+Kafka	Five systems means five backup/consistency/ops stories before the product works. pgvector + tsvector cover both search modes in one transaction domain. Interfaces keep the swap cheap when a measured limit appears. Trade-off: pgvector ANN indexes trail dedicated engines at very large scale.
Vertical slice, then phases	A thin end-to-end thread beats ten half-modules: every phase ships into a working system, and integration risk is paid continuously instead of at the end.
LLM listwise reranker instead of a cross-encoder	No torch, no native binaries (the dev machine's security policy blocks unsigned DLLs), competitive quality, one less model to host. Trade-off: an extra LLM call's latency and cost per request.
Heuristic code chunker instead of tree-sitter	tree-sitter is a native extension - same DLL-policy problem. Regex-based top-level boundary detection gets most of the value; the function signature is tree-sitter-compatible for a later swap.
Server-side web tools instead of a Playwright browser agent	Web-grounded answers with citations, zero browser infrastructure, no bot-detection arms race. Full browser automation deferred - it needs its own security design (sandboxing, action confirmation).
Memory extraction as a background task	An LLM extraction call would add a full round-trip to every chat response. Post-response extraction costs nothing user-visible; failures log and never raise. Uses a strong-reference task set to avoid asyncio garbage collection of fire-and-forget tasks.
Auth optional (dev mode)	Single-user development friction matters; the same dependency enforces JWT everywhere the moment AUTH_SECRET is set.
create_all now, Alembic later	Migrations formalize a schema; formalizing one that is still moving creates churn. The cutover point is documented.
docker-compose, not Kubernetes	Compose runs the full stack on a laptop. Kubernetes before users is operational cost with no payoff.
Graded answers over silent answers	Every RAG answer carries a machine-generated confidence score and unsupported-claims list - hallucination risk is made visible to the caller and trackable on a dashboard.

9. Engineering War Stories (real problems hit and fixed)

9.1 The blocked native DLL

The development machine enforces a Windows Application Control policy that blocks unsigned native extensions. langchain-core (a LangGraph dependency) unconditionally imports uuid_utils - a Rust extension - purely for uuid7, which made the entire app unimportable. Fix: src/nexus/_compat.py pre-registers a pure-Python stand-in module in sys.modules (delegating to the Python 3.14 stdlib uuid7, with an RFC 9562 fallback for older versions) only when the real package fails to import; healthy machines never activate it. The langsmith pytest plugin needed disabling for the same reason (-p no:langsmith_plugin in pyproject).

9.2 The chunker default that failed valid calls

chunk_text(text, max_chars=120) crashed: the default overlap (200) exceeded the caller's max_chars and hit a validation guard. The unit tests caught it immediately. The fix encodes an API-design principle: a default must never make an otherwise valid call fail - the overlap now clamps to max_chars/4 only in the collision case, preserving explicit valid choices.

9.3 The asyncio fire-and-forget trap

asyncio.create_task returns a task the event loop holds only weakly; without a strong reference it can be garbage collected mid-flight. The post-stream memory write uses the standard pattern: a module-level set holds each task, and a done callback discards it.

9.4 pause_turn on server-side tools

Claude's server-side tool loop can pause a long research turn (stop_reason 'pause_turn'). Naive code would return a truncated answer. The provider appends the partial assistant turn and re-sends, letting the server resume, bounded by a max-continuations limit.

10. API Endpoint Reference

Endpoint	Method	Purpose
/health	GET	Liveness check (no auth).
/chat	POST	Full pipeline; returns answer, route, memories, sources, confidence, unsupported claims.
/chat/stream	POST	Same pipeline over SSE: meta, route, sources, delta*, grade, done.
/documents	POST	Ingest a text document (chunk, embed, store).
/repos	POST	Ingest a local repository as kind='code'.
/vision/analyze	POST	Analyze an uploaded image; optionally ingest the analysis.
/memories/search	GET	Semantic search over stored memories.
/conversations	GET	List conversations with message counts.
/conversations/{id}	GET / DELETE	Full transcript / delete with cascade.
/metrics	GET	Prometheus metrics (no auth).
/docs	GET	Auto-generated interactive OpenAPI documentation.

11. Anticipated Interview Questions (with answers)

Q: Why LangGraph and not just plain Python functions or LangChain chains?

A: The pipeline needs conditional branching (three routes), shared typed state, and mid-node streaming - and it will need loops (e.g. retry on low confidence). LangGraph models exactly that as a compiled state machine; chains are linear and hand-rolled orchestration reinvents state management, streaming plumbing, and conditional wiring badly. LangChain itself appears only as LangGraph's core dependency.

Q: Why not a dedicated vector database?

A: pgvector inside Postgres serves cosine similarity search alongside relational data and full-text search in one transaction domain, one backup story, one operational surface. Dedicated engines win at very large scale (billions of vectors, high QPS ANN) - the storage access is behind small functions precisely so that swap is cheap when measurements justify it.

Q: Explain RRF. Why not normalize scores and add them?

A: Cosine similarity and `ts_rank` live on different, distribution-dependent scales; normalizing them robustly is fragile. RRF ignores scores entirely: each item earns $\text{sum}(1/(k+\text{rank}))$ over the lists it appears in ($k=60$). It is rank-based, parameter-light, and rewards agreement between retrievers - the standard fusion baseline that is hard to beat.

Q: How do you stop the model from hallucinating?

A: Three layers: the prompt instructs answering only from memories/excerpts with citations and admitting ignorance; retrieval quality is maximized (rewrite + hybrid + rerank); and a separate LLM judge grades every answer against the context, returning a confidence score and explicit unsupported claims - tracked in a Prometheus histogram so degradation is visible.

Q: What happens when the LLM returns garbage JSON?

A: It mostly cannot - structured outputs constrain generation to a strict schema. But every payload still passes through a pure defensive parser (clamp scores, drop invalid entries, repair permutations, default to safe values), and every auxiliary stage degrades: rewrite falls back to the original question, rerank to fused order, grading to 'ungraded', memory extraction to 'nothing saved'. A model failure can never 500 a chat request.

Q: How does memory work and why does it not slow down chat?

A: After the response is sent, a background task asks Claude to extract self-contained memories (typed episodic/semantic/procedural) from the turn under a strict schema with explicit 'empty list is a good answer' instructions. They are embedded and stored; recall is cosine search that also bumps access stats (the groundwork for decay). Because it is post-response and exception-swallowing, it adds zero latency and zero failure modes to chat.

Q: How do you know your retrieval is any good?

A: An offline golden set maps questions to the documents that should answer them; the harness runs real retrieval and reports `hit@k` and MRR. It runs before and after any change to chunking, embeddings, rewriting, fusion, or reranking - retrieval changes are measured, not vibes-checked. Seeded demo data gives a baseline on a fresh clone with no API keys.

Q: Walk me through the streaming implementation.

A: The Anthropic SDK streams token deltas; the generate node forwards each delta into LangGraph's custom stream channel via `get_stream_writer()`. The endpoint consumes `graph.astream(stream_mode=['updates','custom'])`: node updates become route/sources/grade SSE events, custom items become delta events, and a final done event carries the assembled payload. Invoked non-streaming, the same writes are dropped - one pipeline serves both endpoints.

Q: How would you scale this?

A: In order of measured need: HNSW index on the vector columns (currently exact scan); read replicas for retrieval; move background memory writes to a real queue/worker; cache embeddings for repeated queries; split

the research agent out if web latency dominates; then consider a dedicated vector store. The metrics endpoints already expose the numbers those decisions need.

Q: What is your testing strategy?

A: Extract anything decidable into pure functions and test those exhaustively - chunkers, fusion, every parser, metrics, auth, SSE formatting: 53 tests, ~1 second, no network or database. I/O layers stay thin and are covered by import/startup verification and CI. The suite caught a real API-design bug (chunker overlap default) during development.

Q: What was the hardest bug?

A: The whole app failing to import on a locked-down Windows machine because langchain-core unconditionally imports a Rust extension (uuid_utils) that the Application Control policy blocked. Diagnosed via the import chain, fixed with a conditional pure-Python sys.modules shim that only activates when the real package cannot load - plus disabling the langsmith pytest plugin that imports the same library.

Q: What are the current limitations?

A: Single-tenant (JWT identifies but does not partition data); exact-scan vector search (fine at this scale, needs an HNSW index later); create_all instead of Alembic migrations; memory decay designed but not yet active; no browser automation; the LLM reranker and judge add latency and cost that a busy deployment would tune with the existing feature toggles.

12. Glossary

Term	Meaning
LLM	Large language model - a neural network trained on text to predict tokens; Claude, GPT, etc.
Token	The sub-word unit LLMs read and write (~3/4 of an English word). Pricing and context limits are in tokens.
Context window	The maximum tokens a model can consider per request (prompt + response).
System prompt	Operator instructions that frame the model's behavior for a request, separate from user messages.
Adaptive thinking	Claude feature: the model decides when/how much to reason internally before answering.
Embedding	A vector representation of text where semantic similarity becomes geometric closeness.
Cosine similarity	Similarity of two vectors by the angle between them; the standard embedding comparison.
Vector database	Storage optimized for nearest-neighbor search over embeddings (here: pgvector inside Postgres).
RAG	Retrieval-Augmented Generation - retrieve relevant snippets, put them in the prompt, answer from them.
Chunking	Splitting documents into retrieval-sized pieces.
Hybrid search	Combining semantic (vector) and lexical (keyword) retrieval.
RRF	Reciprocal Rank Fusion - merging ranked lists using ranks, not scores.
Reranking	Second-stage reordering of retrieved candidates for precision.
LLM-as-judge	Using a model call to evaluate another model's output (here: groundedness grading).
Structured outputs	API-enforced JSON-schema conformance of model responses.
Agent	An LLM system that takes actions (tool calls, routing decisions), not just single completions.
Orchestrator	The component sequencing multi-step LLM workflows (here: a LangGraph state machine).
SSE	Server-Sent Events - one-way HTTP streaming used for token-by-token responses.
JWT	JSON Web Token - a signed, stateless authentication token.
ORM	Object-Relational Mapper - database rows as typed objects (SQLAlchemy).

Term	Meaning
hit@k / MRR	Retrieval metrics: correct-source-in-top-k rate; mean reciprocal rank of the first correct result.
Prometheus / Grafana	Metrics collection via scraped /metrics endpoints; dashboarding on top.
CI/CD	Automated lint/test (and deploy) pipelines on every push - here GitHub Actions.
pause_turn	Claude stop reason meaning a server-side tool loop paused; re-send the turn to resume.